

计算机系统设计PA1实验报告

苏耀磊 2311727

一、环境概述

使用 vscode 连接 docker 容器，在 debian 中进行实验，实验指导书参考ics2017。

二、实验内容

- 1.实现正确的寄存器结构体
- 2.实现单步执行, 打印寄存器, 扫描内存
 - 2.1 实现单步执行
 - 2.2 实现寄存器打印
 - 2.3 实现扫描内存
- 3.表达式求值
 - 3.1 词法分析
 - 3.2 递归求值
 - 3.3 实现带负数的表达式求值
 - 3.4 实现更复杂的表达式求值
- 4.监视点
 - 4.1 实现监视点池的管理
 - 4.2 实现监视点
 - 4.3 调试器命令集成
 - 4.4 断点
- 5.i386手册

三、实验过程

1.实现正确的寄存器结构体

在 PA0 时运行 NEMU 会产生 assertion fail 的错误，是因为我们的框架代码中没有正确地实现用于模拟寄存器的结构体CPU_state，查阅i386手册，寄存器结构大概是这样

GENERAL REGISTERS

31	23	15	7	0	
		EAX	AH	AX	AL
		EDX	DH	DX	DL
		ECX	CH	CX	CL
		EBX	BH	BX	BL
		EBP		BP	
		ESI		SI	
		EDI		DI	
		ESP		SP	

i386 通用寄存器在指令编码中有固定索引（如 000 对应 A 组寄存器，011 对应 B 组寄存器），并且存在 32 位、16 位和 8 位别名关系：

- 32 位：`eax`, `ecx`, `edx`, `ebx`, `esp`, `ebp`, `esi`, `edi`
- 16 位：`ax`, `cx`, `dx`, `bx`, `sp`, `bp`, `si`, `di`
- 8 位：低 8 位 `al`, `cl`, `dl`, `bl` 与高 8 位 `ah`, `ch`, `dh`, `bh`

因此结构体必须同时满足：

1. 可按“寄存器编号”索引访问。
2. 可按“寄存器名字”直接访问。
3. 同一物理寄存器的不同宽度视图必须共享同一片内存。

而根据提示采用 union 结构，代码实现如下：

```

typedef struct {
    union {
        union {
            uint32_t _32;
            uint16_t _16;
            uint8_t _8[2];
        } gpr[8];

        /* Do NOT change the order of the GPRs' definitions. */

        /* In NEMU, rtlreg_t is exactly uint32_t. This makes RTL instructions
         * in PA2 able to directly access these registers.
         */
        struct {
            rtlreg_t eax, ecx, edx, ebx, esp, ebp, esi, edi;
        };
    };
    vaddr_t eip;
} CPU_state;

```

实现的核心是两个 `union` 叠加：

- 外层 `union` 让 `gpr[8]` 数组视图与 `eax...edi` 命名视图共址。
- 内层 `union` 让 `_32/_16/_8[2]` 三种宽度共址。

这意味着：

- 写 `cpu.eax` 会同步改变 `cpu.gpr[R_EAX]._32`。
- 写 `cpu.gpr[R_EBX]._16` 本质在改 `bx` 子寄存器。
- 读 `cpu.gpr[1]._8[1]` 获得 `ch`（即 `ecx` 的第 8~15 位）。

这样实现以后，我们执行 `make run`，得到结果如下：

```

SuYaoLei@988d7f56373a:~/code/ics2017/nemu$ make run
./build/nemu -l ./build/nemu-log.txt
[src/monitor/monitor.c,47,load_default_img] No image is given. Use the default build-in image.
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 16:10:27, Apr  9 2026
For help, type "help"
(nemu) c
nemu: HIT GOOD TRAP at eip = 0x00100026

(nemu) █

```

成功实现。

问题：究竟要执行多久？

在 `cmd_c()` 函数中，调用 `cpu_exec()` 的时候传入了参数 -1，你知道这是什么意思吗？

解答：

我们可以看一下`cpu_exec()`函数的实现：

```
void cpu_exec(uint64_t n) {
    if (nemu_state == NEMU_END) {
        printf("Program execution has ended. To restart the program, exit NEMU and run again.\n");
        return;
    }
    nemu_state = NEMU_RUNNING;

    bool print_flag = n < MAX_INSTR_TO_PRINT;

    for (; n > 0; n --) {
        /* Execute one instruction, including instruction fetch,
         * instruction decode, and the actual execution. */
        exec_wrapper(print_flag);

#ifdef DEBUG
        /* TODO: check watchpoints here. */

        if (check_watchpoints()) {
            nemu_state = NEMU_STOP;
        }
#endif

#ifdef HAS_IOE
        extern void device_update();
        device_update();
#endif

        if (nemu_state != NEMU_RUNNING) { return; }
    }

    if (nemu_state == NEMU_RUNNING) { nemu_state = NEMU_STOP; }
}
```

可以看到，`cpu_exec()`函数接受的是一个无符号整数`uint64_t`，而当我们传入一个负数后，按照c语言的规则会将其转换为无符号整数，按照“模 2^{64} ”进行转换，也就是：

```
(uint64_t)(-1) == 2^64 - 1
/*18446744073709551615*/
```

所以按照c语言的执行方式来说，它实际上执行了非常非常多的指令，也就是一直执行到结束。

2.实现单步执行, 打印寄存器, 扫描内存

在 `nemu/src/monitor/debug/ui.c` 中，我们需要自己实现一些调试工具，具体根据指导书的内容如下：

命令	格式	使用举例	说明
帮助(1)	<code>help</code>	<code>help</code>	打印命令的帮助信息
继续运行(1)	<code>c</code>	<code>c</code>	继续运行被暂停的程序
退出(1)	<code>q</code>	<code>q</code>	退出NEMU
单步执行	<code>si [N]</code>	<code>si 10</code>	让程序单步执行 <code>N</code> 条指令后暂停执行, 当 <code>N</code> 没有给出时, 缺省为 <code>1</code>
打印程序状态	<code>info SUBCMD</code>	<code>info r</code> <code>info w</code>	打印寄存器状态 打印监视点信息
表达式求值	<code>p EXPR</code>	<code>p \$eax + 1</code>	求出表达式 <code>EXPR</code> 的值, <code>EXPR</code> 支持的运算请见 调试中的表达式求值 小节
扫描内存(2)	<code>x N EXPR</code>	<code>x 10 \$esp</code>	求出表达式 <code>EXPR</code> 的值, 将结果作为起始内存地址, 以十六进制形式输出连续的 <code>N</code> 个4字节
设置监视点	<code>w EXPR</code>	<code>w *0x2000</code>	当表达式 <code>EXPR</code> 的值发生变化时, 暂停程序执行
删除监视点	<code>d N</code>	<code>d 2</code>	删除序号为 <code>N</code> 的监视点

2.1 实现单步执行

`si` 命令其实就是“执行有限条指令再停下”，因此实现选择直接复用执行引擎接口 `cpu_exec(n)`：无参数时执行 1 条，符合调试器直觉；有参数时将其解释为步数，非法值 (`<=0`) 直接拒绝。

具体实现如下：

```

39  static int cmd_si(char *args) {
40      if (args == NULL) {
41          cpu_exec(1);
42      }
43      else {
44          int n = atoi(args);
45          if (n <= 0) {
46              printf("Invalid number of instructions: %d\n", n);
47              return 0;
48          }
49          cpu_exec(n);
50      }
51      return 0;
52  }

```

进行测试，得到结果如下：

```
SuYao1ei@988d7f56373a:~/code/ics2017/nemu$ make run
./build/nemu -l ./build/nemu-log.txt
[src/monitor/monitor.c,47,load_default_img] No image is given. Use the default build-in image.
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 15:29:51, Apr 11 2026
For help, type "help"
(nemu) si
    100000:  b8 34 12 00 00          movl $0x1234,%eax
(nemu) si 3
    100005:  b9 27 00 10 00          movl $0x100027,%ecx
    10000a:  89 01                   movl %eax,(%ecx)
    10000c:  66 c7 41 04 01 00       movw $0x1,0x4(%ecx)
(nemu) █
```

这说明实现是正确的。之后还需要将我们实现的命令补充到 `cmd_table` 中，使得我们能够在命令行使用它们。

命令行通过 `strtok(str, " ")` 来解析我们的命令，并调用 `cmd_table[i].handler(args)` 来执行具体是哪一个函数，查找 `cmd_table` 从而得到我们刚才设计的函数，比如 `cmd_si`，补充的方法如下：

```
197 static struct {
198     char *name;
199     char *description;
200     int (*handler) (char *);
201 } cmd_table [] = {
202     { "help", "Display informations about all supported commands", cmd_help },
203     { "c", "Continue the execution of the program", cmd_c },
204     { "q", "Exit NEMU", cmd_q },
205
206     /* TODO: Add more commands */
207     { "si", "Step into instruction(s)", cmd_si },
208     { "info", "Display register/watchpoint status", cmd_info },
209     { "x", "Examine memory", cmd_x },
210     { "p", "Evaluate expression: p EXPR", cmd_p },
211     { "w", "Set a watchpoint: w EXPR", cmd_w },
212     { "d", "Delete a watchpoint: d N", cmd_d },
213 };
```

这里也补充了其他的一些命令，因此在下文不再重复。

2.2 实现寄存器打印

`info r` 直接使用 `printf` 读取 `CPU_state` 的命名字段打印，而 `info w` 调用 `list_watchpoints()`，看观察点列表，`list_watchpoints()` 的实现会在后面介绍。

代码实现如下：

```
54 static int cmd_info(char *args) {
55     if (args == NULL) {
56         printf("Usage: info [r/w]\n");
57         return 0;
58     }
59
60     if (strcmp(args, "r") == 0) {
61         printf("Registers:\n");
62         printf("eax: 0x%08x\n", cpu.eax);
63         printf("ecx: 0x%08x\n", cpu.ecx);
64         printf("edx: 0x%08x\n", cpu.edx);
65         printf("ebx: 0x%08x\n", cpu.ebx);
66         printf("esp: 0x%08x\n", cpu.esp);
67         printf("ebp: 0x%08x\n", cpu.ebp);
68         printf("esi: 0x%08x\n", cpu.esi);
69         printf("edi: 0x%08x\n", cpu.edi);
70         printf("eip: 0x%08x\n", cpu.eip);
71     }
72     else if (strcmp(args, "w") == 0) {
73         list_watchpoints();
74     }
75     else {
76         printf("Unknown info command '%s'\n", args);
77     }
78     return 0;
79 }
80
```

测试结果：

```
SuYao1ei@988d7f56373a:~/code/ics2017/nemu$ make run
./build/nemu -l ./build/nemu-log.txt
[src/monitor/monitor.c,47,load_default_img] No image is given. Use the default build-in image.
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 15:29:51, Apr 11 2026
For help, type "help"
(nemu) info r
Registers:
eax: 0x170c93c8
ecx: 0x02802f63
edx: 0x3ab974d6
ebx: 0x3f99fdd5
esp: 0x78f008b6
ebp: 0x436a357f
esi: 0x1fb56d12
edi: 0x5425252f
eip: 0x00100000
(nemu) █
```

实现没有问题。

2.3 实现扫描内存

按实验要求，我们设置支持十六进制常量地址。先进行一系列命令的正确性检测，包括参数是否存在，参数个数是否合法，N是不是合法的正整数以及EXPR是不是合法的十六进制地址，从而避免执行非法命令，后面调用 `vaddr_read(cur_addr, 4)`，每次读取 4 字节，但是读取的是“虚拟地址空间”而非物理地址。

代码实现如下：

```

133 static int cmd_x(char *args) {
134     if (args == NULL) {
135         printf("Usage: x N EXPR\n");
136         return 0;
137     }
138
139     char *n_str = strtok(args, " ");
140     char *expr_str = strtok(NULL, " ");
141
142     if (n_str == NULL || expr_str == NULL) {
143         printf("Usage: x N EXPR\n");
144         return 0;
145     }
146
147     char *n_end = NULL;
148     long n = strtol(n_str, &n_end, 10);
149     if (*n_str == '\0' || *n_end != '\0' || n <= 0) {
150         printf("Invalid number of words: %s\n", n_str);
151         return 0;
152     }
153
154     char *expr_end = NULL;
155     uint32_t addr = strtoul(expr_str, &expr_end, 16);
156     if (*expr_str == '\0' || *expr_end != '\0') {
157         printf("EXPR must be a hexadecimal number, e.g. 0x100000\n");
158         return 0;
159     }
160
161     for (int i = 0; i < n; i++) {
162         uint32_t cur_addr = addr + i * 4;
163         uint32_t data = vaddr_read(cur_addr, 4);
164         printf("0x%08x: 0x%08x\n", cur_addr, data);
165     }
166
167     return 0;
168 }

```

测试得到：

```

❖ SuYaolei@988d7f56373a:~/code/ics2017/nemu$ make run
./build/nemu -l ./build/nemu-log.txt
[src/monitor/monitor.c,47,load_default_img] No image is given. Use the default build-in image.
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 15:29:51, Apr 11 2026
For help, type "help"
(nemu) x 5 0x100000
0x00100000: 0x001234b8
0x00100004: 0x0027b900
0x00100008: 0x01890010
0x0010000c: 0x0441c766
0x00100010: 0x02bb0001
(nemu) x 1 0x100014
0x00100014: 0x66000000
(nemu) █

```

这说明实现是正确的。

3.表达式求值

对于表达式求值，按照指导书说明，我们可以分成两个部分：

1. 首先识别出表达式中的单元
2. 根据表达式的归纳定义进行递归求值

3.1 词法分析

首先是词法分析，表达式词法分析位于 `nemu/src/monitor/debug/expr.c`，其目标是把原始字符串变成统一 token 序列，为后续递归求值提供结构化输入。

指导书的表达式满足如下，因此我们需要补充对应的token和相应的规则。

```

<expr> ::= <decimal-number>
| <hexadecimal-number>      # 以"0x"开头
| <reg_name>                 # 以"$"开头
| "(" <expr> ")"
| <expr> "+" <expr>
| <expr> "-" <expr>
| <expr> "*" <expr>
| <expr> "/" <expr>
| <expr> "==" <expr>
| <expr> "!=" <expr>
| <expr> "&&" <expr>
| <expr> "||" <expr>
| "!" <expr>
| "*" <expr>                 # 指针解引用

```

token 类型定义如下，覆盖了数值、寄存器、比较逻辑运算符以及一元运算符：

```
11  ✓ enum {
12      TK_NOTYPE = 256,
13      TK_EQ,
14      TK_NEQ,
15      TK_AND,
16      TK_OR,
17      TK_NOT,
18      TK_NEG,          //一元负号
19      TK_DEREF,        //一元*解引用
20      TK_DEC,
21      TK_HEX,
22      TK_REG
23
24      /* TODO: Add more token types */
25
26  };
```

对应规则为：

```

28  static struct rule {
29      char *regex;
30      int token_type;
31  } rules[] = {
32
33      /* TODO: Add more rules.
34       * Pay attention to the precedence level of different rules.
35       */
36
37      {" +", TK_NOTYPE},           // spaces
38      {"==" , TK_EQ},             // equal
39      {"!=" , TK_NEQ},           // not equal
40      {"&&" , TK_AND},           // and
41      {"\\|\\|" , TK_OR},        // or
42      {"!" , TK_NOT},            // not
43      {"\\+" , '+'},             // plus
44      {"\\-" , '-'},             // minus
45      {"\\*" , '*'},             // multiply
46      {"\\/" , '/'},             // divide
47      {"\\(" , '('},             // left parenthesis
48      {"\\)" , ')'},             // right parenthesis
49      {"0[xX][0-9a-fA-F]+" , TK_HEX}, // hex number
50      {"[0-9]+" , TK_DEC},        // decimal number
51      {"\\$[a-zA-Z][a-zA-Z0-9]*" , TK_REG} // register
52  };

```

词法实现使用 POSIX regex ([regcomp/regexec](#)) 逐位置匹配，若在当前位置没有任何规则命中，则立即报错并返回失败，从而尽早暴露非法输入，避免把错误延迟到求值阶段。

在 token 记录策略上，数值与寄存器 token 保留 `str` 字段，运算符只保存 `type`。这也是为什么调试输出里运算符 `str` 可能为空，但计算仍然正确：求值只依赖 `type`，不依赖运算符字符串。

之后是在 `make_token` 函数里对各类 token 进行处理，将 token 的信息依次记录到 `tokens` 数组。

下面是 `make_token()` 中“将识别结果写入 `tokens` 数组”的核心代码：

```

106  switch (rules[i].token_type) {
107      case TK_NOTYPE:
108          break;
109
110      case TK_DEC:
111      case TK_HEX:
112      case TK_REG:
113          if (nr_token >= (int)(sizeof(tokens) / sizeof(tokens[0]))) {
114              printf("too many tokens\n");
115              return false;
116          }
117          if (substr_len >= (int)sizeof(tokens[nr_token].str)) {
118              printf("token is too long: %.*s\n", substr_len, substr_start);
119              return false;
120          }
121          tokens[nr_token].type = rules[i].token_type;
122          strncpy(tokens[nr_token].str, substr_start, substr_len);
123          tokens[nr_token].str[substr_len] = '\0';
124          nr_token++;
125          break;
126
127      default:
128          if (nr_token >= (int)(sizeof(tokens) / sizeof(tokens[0]))) {
129              printf("too many tokens\n");
130              return false;
131          }
132          tokens[nr_token].type = rules[i].token_type;
133          tokens[nr_token].str[0] = '\0';
134          nr_token++;
135          break;
136  }

```

这里空白符（TK_NOTYPE）在表达式语义上没有贡献，所以直接丢弃；十进制、十六进制与寄存器属于后续求值必须读取字面文本的 token（例如要把 "123" 转成整数，或把 "\$eax" 映射到寄存器值），因此除了记录 **type**，还要把子串复制进 **str**；其余运算符 token（如 + - * / == &&）后续只用 **type** 判定运算语义，不需要额外文本，故把 **str** 置空即可。

接下来我们需要做一些边界防护，**nr_token** 上限检查用于防止表达式过长导致越界写入 **tokens**；**substr_len** 长度检查用于防止字面量过长写爆 **Token.str**。

对需要保存文本的 token，代码在 **strncpy** 后手动补 **\0**，确保 C 字符串总是可安全打印/比较；每成功记录一个 token 后执行 **nr_token++**，保持“数组前缀 **[0, nr_token)** 始终有效”的不变式。这个不变式非常重要，因为后续 **eval()**、一元运算重写、调试输出都依赖它进行线性扫描。

处理完这些后，需要另外注意的是 **-** 与 ***** 在表达式中既可能是二元运算符，也可能是一元运算符，因此我选择在词法结束后做一次 token 重写。

```

148  ∨  for (i = 0; i < nr_token; i++) {
149      //对于每一个-, 若在表达式开头, 或前一个 token 不是“操作数/右括号”, 改写为 TK_NEG
150  ∨      if (tokens[i].type == '-') {
151  ∨          if (i == 0 ||
152  ∨              !(tokens[i - 1].type == TK_DEC || tokens[i - 1].type == TK_HEX ||
153  ∨                  tokens[i - 1].type == TK_REG || tokens[i - 1].type == ')')) {
154              tokens[i].type = TK_NEG;
155          }
156      }
157      //对每个 *, 若在表达式开头, 或前一个 token 不是“操作数/右括号”, 改写为 TK_DEREF
158  ∨      else if (tokens[i].type == '*') {
159  ∨          if (i == 0 ||
160  ∨              !(tokens[i - 1].type == TK_DEC || tokens[i - 1].type == TK_HEX ||
161  ∨                  tokens[i - 1].type == TK_REG || tokens[i - 1].type == ')')) {
162              tokens[i].type = TK_DEREF;
163          }
164      }
165  }

```

这里实现的方式是若某个 - 或 * 前面不是“可结束子表达式的 token”(数字、寄存器、右括号), 就将其视为前缀一元运算符。

到这里, 我们就完成了算术表达式的词法分析。

3.2 递归求值

词法分析完成后, 我们得到了一个 token 序列, 接下来我们需要根据这个序列进行递归求值, 求值函数采用经典分治框架, 入口为 `expr()`, 核心为 `eval(p, q, success)`。关键流程如下:

- `p > q`: 非法表达式;
- `p == q`: 处理原子项 (十进制、十六进制、寄存器);
- `check_parentheses(p, q)`: 若整段被外层一对括号包裹, 则剥壳递归;
- 否则在当前层 (`level == 0`) 扫描主运算符并递归求左右值。

比较关键的两个点就是括号匹配以及主运算符扫描, 下面分别介绍。

先看括号匹配函数:

```

206 static bool check_parentheses(int p, int q) {
207     int i;
208     int level = 0;
209
210     if (tokens[p].type != '(' || tokens[q].type != ')') {
211         return false;
212     }
213
214     for (i = p; i <= q; i++) {
215         if (tokens[i].type == '(') {
216             level++;
217         }
218         else if (tokens[i].type == ')') {
219             level--;
220             if (level == 0 && i < q) {
221                 return false;
222             }
223             if (level < 0) {
224                 return false;
225             }
226         }
227     }
228
229     return level == 0;
230 }

```

整个代码的逻辑是接收两个整数参数 p 和 q，分别表示要检查的起始和结束位置。

首先，函数会判断区间两端的 token 类型是否分别是左括号 (和右括号)。如果不是，则说明整个区间没有被完整的括号包围，函数立即返回 false。

接着，函数使用一个 level 变量来追踪括号的嵌套层数，并通过一个循环从 p 到 q 遍历每个 token。当遇到左括号时，level 加一；遇到右括号时，level 减一。如果在遍历过程中 level 变为 0 且当前位置还没有到达区间的末尾 q，说明外层括号提前闭合，区间中还有多余的内容未被外层括号完全包裹，因此返回 false。如果 level 小于 0，则说明出现了右括号多于左括号的情况，括号不匹配，也返回 false。

最后，循环结束后，函数通过 return level == 0; 来判断所有括号是否完全配对。如果 level 为 0，说明括号完全匹配且整个区间被外层括号包围，函数返回 true；否则返回 false。

而寻找主运算符部分的代码如下：

```

281  ✓   for (i = p; i <= q; i++) {
282  ✓       if (tokens[i].type == '(') {
283           level++;
284           continue;
285       }
286  ✓       if (tokens[i].type == ')') {
287           level--;
288  ✓       if (level < 0) {
289           *success = false;
290           return 0;
291       }
292       continue;
293     }
294
295  ✓     if (level == 0) {
296         int cur_prec = precedence(tokens[i].type);
297  ✓     if (cur_prec <= 5 && cur_prec <= op_prec) {
298         op_prec = cur_prec;
299         op = i;
300     }
301     }
302 }

```

其实就是在当前表达式层选择最低优先级的可分割点，具体来说，`level` 的作用是屏蔽子表达式内部运算符。每遇到 `(` 就进入更深层，每遇到 `)` 就返回上一层，只有 `level == 0` 的位置才属于当前层运算，这样 `(1+2)*3` 中的 `+` 会被跳过，`*` 才可能成为主运算符。

之后我们要比较 `cur_prec <= op_prec`，保证取更低优先级以及同优先级取最右，配合递归切分 `eval(p, op-1)` 与 `eval(op+1, q)`，能正确处理左结合运算。例如 `1-2-3` 扫描后主运算符落在第二个 `-`，递归结果是 `(1-2)-3`。

最后通过 `if (level != 0) { *success = false; return 0; }` 来进行一致性检查保证扫描结束时括号层级必须回到 0，否则说明括号不匹配，直接判错，避免后续在错误结构上继续递归。

`precedence` 是运算符的优先级设置函数，如下：

```

232  ∨ static int precedence(int type) {
233  ∨      switch (type) {
234          case TK_OR: return 1;
235          case TK_AND: return 2;
236          case TK_EQ:
237          case TK_NEQ: return 3;
238          case '+':
239          case '-': return 4;
240          case '*':
241          case '/': return 5;
242          default: return 100;
243      }
244  }

```

优先级函数设置为：`|| < && < ==/!= < +/ - < */`，扫描时使用 `<=` 选择同优先级中更靠右的运算符。

扫描结束后若 `op == -1`，说明当前区间不存在二元主运算符，表达式可能是单目运算符，如负号 `TK_NEG`、逻辑非 `TK_NOT` 或指针解引用 `TK_DEREF`。此时，函数递归求值操作符右边的子表达式，再进行相应的单目操作。如果子表达式求值失败，函数直接返回 0：

```

309     if (op == -1) {
310         if (tokens[p].type == TK_NEG) {
311             uint32_t val = eval(p + 1, q, success);
312             if (!*success) {
313                 return 0;
314             }
315             return (uint32_t)(-(int32_t)val);
316         }
317         else if (tokens[p].type == TK_NOT) {
318             uint32_t val = eval(p + 1, q, success);
319             if (!*success) {
320                 return 0;
321             }
322             return !val;
323         }
324         else if (tokens[p].type == TK_DEREF) {
325             uint32_t addr = eval(p + 1, q, success);
326             if (!*success) {
327                 return 0;
328             }
329             return vaddr_read(addr, 4);
330         }
331
332         *success = false;
333         return 0;
334     }

```

这里涉及两个重要的点：

1. `TK_NEG` 采用 `-(int32_t)val` 后再转回 `uint32_t`，因为 i386 对负数运算本质是按固定位宽二进制补码执行，转换后再以无符号显示并不改变底层比特。
2. `TK_DEREF` 使用 `vaddr_read(addr, 4)`，表示按 32 位虚拟地址读取一个 word，与 i386 32 位模式中的通用 dword 访存模型一致。

最后，如果找到了主要操作符 `op`，函数将表达式分为左右两个子区间，递归计算左右子表达式的值。然后根据操作符类型执行对应的运算，包括加减乘除、相等与不等、逻辑与和逻辑或。特别地，对于除法，会检查除数是否为 0，以避免运行时错误。如果操作符未知，则返回失败。代码实现如下：

```

336     uint32_t val1 = eval(p, op - 1, success);
337     if (!*success) {
338         return 0;
339     }
340     uint32_t val2 = eval(op + 1, q, success);
341     if (!*success) {
342         return 0;
343     }
344
345     switch (tokens[op].type) {
346         case '+': return val1 + val2;
347         case '-': return val1 - val2;
348         case '*': return val1 * val2;
349         case '/':
350             if (val2 == 0) {
351                 *success = false;
352                 return 0;
353             }
354             return val1 / val2;
355         case TK_EQ: return val1 == val2;
356         case TK_NEQ: return val1 != val2;
357         case TK_AND: return val1 && val2;
358         case TK_OR: return val1 || val2;
359         default:
360             *success = false;
361             return 0;
362     }

```

3.3 实现带负数的表达式求值

在之前的部分也基本介绍了如何实现负数的运算，代码如下：

```

148     for (i = 0; i < nr_token; i++) {
149         //对于每一个-, 若在表达式开头, 或前一个 token 不是“操作数/右括号”, 改写为 TK_NEG
150         if (tokens[i].type == '-') {
151             if (i == 0 ||
152                 !(tokens[i - 1].type == TK_DEC || tokens[i - 1].type == TK_HEX ||
153                   tokens[i - 1].type == TK_REG || tokens[i - 1].type == ')')) {
154                 tokens[i].type = TK_NEG;
155             }
156         }
157         //对每个 *, 若在表达式开头, 或前一个 token 不是“操作数/右括号”, 改写为 TK_DEREF
158         else if (tokens[i].type == '*') {
159             if (i == 0 ||
160                 !(tokens[i - 1].type == TK_DEC || tokens[i - 1].type == TK_HEX ||
161                   tokens[i - 1].type == TK_REG || tokens[i - 1].type == ')')) {
162                 tokens[i].type = TK_DEREF;
163             }
164         }
165     }

```

这里再简单补充一下。在之前的词法阶段我已经把语义上属于前缀负号的 `-` 重写为 `TK_NEG`，因此求值阶段不会再把它当成二元减法。进入 `eval()` 后，如果当前区间找不到二元主运算符 (`op == -1`)，且区间起点是 `TK_NEG`，就会递归求值后缀子表达式，然后执行 `-(int32_t)val`。

这意味着负数是通过一元运算语义在求值阶段计算出来的。

以 `1 + -1` 为例，token 会被处理为 `TK_DEC '+' TK_NEG TK_DEC`。主运算符扫描在最外层选中 `+`，左子树是 `1`，右子树是 `-1`。右子树继续进入 `op == -1` 分支，命中 `TK_NEG`，将 `1` 取负后返回。最终结果是 `1 + (-1) = 0`。

以 `--1` 为例，token 为 `TK_NEG TK_NEG TK_DEC`。第一层 `eval` 命中 `TK_NEG`，递归求 `TK_NEG TK_DEC`；第二层再命中 `TK_NEG`，递归求 `TK_DEC` 得到 `1`；回溯时先得 `-1`，再得 `-(-1)=1`。

与此对应，若输入是 `1-1`，词法不会把中间 `-` 重写为 `TK_NEG`，因为它前面是可结束子表达式的数字 token。求值阶段会把它当作二元减法，仍走主运算符分治路径。

3.4 实现更复杂的表达式求值

这里补充的token其实在之前的部分已经介绍过了，这里不再赘述，附上现有的token类型：

```

11  enum {
12      TK_NOTYPE = 256,
13      TK_EQ,
14      TK_NEQ,
15      TK_AND,
16      TK_OR,
17      TK_NOT,
18      TK_NEG,          //一元负号
19      TK_DEREF,        //一元*解引用
20      TK_DEC,
21      TK_HEX,
22      TK_REG
23
24      /* TODO: Add more token types */
25
26  };

```

而如果在你的表达式中存在寄存器的部分，会通过`get_reg_val()`函数来获取寄存器的值，代码如下：

```

257  if (p == q) {
258      if (tokens[p].type == TK_DEC) {
259          return (uint32_t)strtoul(tokens[p].str, NULL, 10);
260      }
261      else if (tokens[p].type == TK_HEX) {
262          return (uint32_t)strtoul(tokens[p].str, NULL, 16);
263      }
264      else if (tokens[p].type == TK_REG) {
265          uint32_t val = 0;
266          if (!get_reg_val(tokens[p].str, &val)) {
267              *success = false;
268              return 0;
269          }
270          return val;
271      }

```

```

170  ✓ static bool get_reg_val(const char *s, uint32_t *val) {
171      int i;
172  ✓  if (s == NULL || s[0] != '$') {
173      |   return false;
174      }
175
176      const char *name = s + 1;
177  ✓  if (strcmp(name, "eip") == 0 || strcmp(name, "pc") == 0) {
178      |   *val = cpu.eip;
179      |   return true;
180      }
181
182  ✓  for (i = 0; i < 8; i++) {
183  ✓  |   if (strcmp(name, regsl[i]) == 0) {
184      |       *val = reg_l(i);
185      |       return true;
186      |   }
187      }
188
189  ✓  for (i = 0; i < 8; i++) {
190  ✓  |   if (strcmp(name, regsw[i]) == 0) {
191      |       *val = reg_w(i);
192      |       return true;
193      |   }
194      }
195
196  ✓  for (i = 0; i < 8; i++) {
197  ✓  |   if (strcmp(name, regsb[i]) == 0) {
198      |       *val = reg_b(i);
199      |       return true;
200      |   }
201      }
202
203      return false;
204  }

```

表达式支持 `$eax`、`$ax`、`$al`、`$eip` 以及 `$pc` 等形式，通过 `get_reg_val()` 在 `regsl/regsw/regsb` 中查找并返回对应宽度的寄存器值。

整个逻辑是在词法阶段把 `$...` 识别为 `TK_REG` 并保留文本后，`eval()` 在单 token 分支中调用该函数完成实际取值。函数先检查输入是否以 `$` 开头，再把寄存器名主体与特殊名（`eip/pc`）和三组标准寄存器名数组进行比

较，匹配后分别通过 `cpu.eip`、`reg_l(i)`、`reg_w(i)`、`reg_b(i)` 返回对应宽度的值。若名称不存在则返回 `false`，上层把本次表达式判为失败。

之后让 `expr()` 来调用 `eval()` 函数，将整个模块的功能集成到一起，代码如下：

```
365  uint32_t expr(char *e, bool *success) {
366      uint32_t result;
367
368      // printf("[expr] input: %s\n", e);
369
370      if (!make_token(e)) {
371          *success = false;
372          return 0;
373      }
374
375      // printf("[expr] tokens (%d):\n", nr_token);
376      // for (int i = 0; i < nr_token; i++) {
377      //     printf("  [%d] type=%d str='%s'\n", i, tokens[i].type, tokens[i].str);
378      // }
379
380      if (nr_token == 0) {
381          *success = false;
382          return 0;
383      }
384
385      *success = true;
386      result = eval(0, nr_token - 1, success);
387      // printf("[expr] success=%d result=0x%08x (%u)\n", *success, result, result);
388      return result;
389  }
```

最后将 `expr()` 集成到调试器中，通过 `cmd_p()` 来调用 `expr()` 函数，使得可以通过 `p` 命令输出求值结果：

```
170  ✓ static int cmd_p(char *args) {
171  ✓     if (args == NULL) {
172      |     printf("Usage: p EXPR\n");
173      |     return 0;
174      | }
175
176  ✓ while (*args == ' ') {
177      |     args++;
178      | }
179  ✓ if (*args == '\\0') {
180      |     printf("Usage: p EXPR\n");
181      |     return 0;
182      | }
183
184     bool success = true;
185     uint32_t result = expr(args, &success);
186  ✓ if (!success) {
187      |     printf("Bad expression: %s\n", args);
188      |     return 0;
189      | }
190
191     printf("%u (0x%08x)\n", result, result);
192     return 0;
193 }
```

进行一下测试：

```

SuYaoLei@988d7f56373a:~/code/ics2017/nemu$ make run
./build/nemu -l ./build/nemu-log.txt
[src/monitor/monitor.c,47,load_default_img] No image is given. Use the default build-in image.
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 19:23:07, Apr 13 2026
For help, type "help"
(nemu) p 6-1*-1
[src/monitor/debug/expr.c,97,make_token] match rules[13] = "[0-9]+" at position 0 with len 1: 6
[src/monitor/debug/expr.c,97,make_token] match rules[7] = "-" at position 1 with len 1: -
[src/monitor/debug/expr.c,97,make_token] match rules[13] = "[0-9]+" at position 2 with len 1: 1
[src/monitor/debug/expr.c,97,make_token] match rules[8] = "*" at position 3 with len 1: *
[src/monitor/debug/expr.c,97,make_token] match rules[7] = "-" at position 4 with len 1: -
[src/monitor/debug/expr.c,97,make_token] match rules[13] = "[0-9]+" at position 5 with len 1: 1
7 (0x00000007)
(nemu) p $eax
[src/monitor/debug/expr.c,97,make_token] match rules[14] = "\${a-zA-Z}[a-zA-Z0-9]*" at position 0 with len 4: $eax
1384646276 (0x52880684)
(nemu) p $pc
[src/monitor/debug/expr.c,97,make_token] match rules[14] = "\${a-zA-Z}[a-zA-Z0-9]*" at position 0 with len 3: $pc
1048576 (0x00100000)
(nemu)

```

这里能看到运算结果是正确的。

4. 监视点

4.1 实现监视点池的管理

监视点结构体在 `nemu/include/monitor/watchpoint.h` 中扩展为：

```

6  typedef struct watchpoint {
7      int NO;
8      struct watchpoint *next;
9
10     /* TODO: Add more members if necessary */
11     char expr[128];
12     uint32_t last_val;
13
14 } WP;
15

```

`NO` 为编号，`next` 组织链表，`expr` 保存表达式文本，`last_val` 保存上一次求值结果。实现采用固定池（`NR_WP=32`）+ 双链表头指针（`head` 与 `free_`）模型：

- `free_` 管理空闲节点；
- `head` 管理正在使用的监视点。

对于 `new_wp()` 与 `free_wp()`，在 `watchpoint.c` 中实现：

```
24  ✓ WP* new_wp() {
25      assert(free_ != NULL);
26
27      WP *wp = free_;
28      free_ = free_->next;
29
30      wp->next = head;
31      head = wp;
32
33      wp->expr[0] = '\0';
34      wp->last_val = 0;
35
36      return wp;
37 }
```

这里`new_wp()`从`free_`取节点并挂入`head`。而`free_wp()`从`head`摘链后归还`free_`，代码实现如下：

```

39 void free_wp(WP *wp) {
40     WP *prev = NULL;
41     WP *cur = head;
42
43     if (wp == NULL) {
44         return;
45     }
46
47     while (cur != NULL && cur != wp) {
48         prev = cur;
49         cur = cur->next;
50     }
51
52     assert(cur != NULL);
53
54     if (prev == NULL) {
55         head = cur->next;
56     }
57     else {
58         prev->next = cur->next;
59     }
60
61     cur->expr[0] = '\0';
62     cur->last_val = 0;
63     cur->next = free_;
64     free_ = cur;
65 }

```

函数定义了两个指针 prev 和 cur，并将 cur 初始化为 watchpoint 使用链表的头节点 head。随后通过 while 循环遍历整个链表，寻找与 wp 相等的节点，找到要释放的节点后，函数根据节点在链表中的位置进行指针调整。如果要释放的节点是头节点，直接将 head 指向其下一个节点；如果是中间或尾节点，则将前一个节点的 next 指针指向被释放节点的下一个节点。随后，函数对节点本身的数据进行清理。

问题：框架代码中定义 wp_pool 等变量的时候使用了关键字 static，static 在此处的含义是什么？为什么要在此处使用它？

static 的作用其实就是限制它的作用域到当前源文件内部，具体来说，这些变量都是模块内部的实现细节，不应该被外部代码直接访问或修改，而如果不加 static，这些变量就变成了全局可见，其他源文件可以随意访问甚至修改它们。

4.2 实现监视点

在 `watchpoint.c` 中，我们需要补充添加监视点的功能 `add_watchpoint()`：

```
67 WP* add_watchpoint(const char *expr_str, bool *success) {
68     assert(expr_str != NULL);
69
70     WP *wp = new_wp();
71     strncpy(wp->expr, expr_str, sizeof(wp->expr) - 1);
72     wp->expr[sizeof(wp->expr) - 1] = '\0';
73
74     wp->last_val = expr(wp->expr, success);
75     if (!*success) {
76         free_wp(wp);
77         return NULL;
78     }
79
80     return wp;
81 }
```

函数首先确保传入的表达式字符串不为空。接着，函数调用 `new_wp()` 创建一个新的 watchpoint 对象，并将其指针保存到 `wp`。

然后，函数使用 `strncpy` 将传入的表达式字符串复制到 watchpoint 的 `expr` 字段，并确保字符串安全截断，防止缓冲区溢出和未终止字符串问题。

接下来，函数通过调用 `expr(wp->expr, success)` 对表达式进行求值，并将结果保存到 watchpoint 的 `last_val` 字段。将解析结果是否成功写入 `*success`。如果表达式解析失败，即 `*success` 为 `false`，函数会调用 `free_wp(wp)` 释放刚刚分配的 watchpoint，并返回 `NULL`，避免创建无效的监视点。

最后，如果表达式解析成功，函数会返回创建好的 watchpoint 指针 `wp`。

接下来是 `delete_watchpoint()` 的实现：

```

83  bool delete_watchpoint(int no) {
84      WP *cur = head;
85
86      while (cur != NULL) {
87          if (cur->NO == no) {
88              free_wp(cur);
89              return true;
90          }
91          cur = cur->next;
92      }
93
94      return false;
95  }

```

`delete_watchpoint(no)` 通过编号查找后调用 `free_wp()` 回收。

还有列举监视点的 `list_watchpoints()`：

```

97  void list_watchpoints(void) {
98      WP *cur = head;
99
100     if (cur == NULL) {
101         printf("No watchpoints.\n");
102         return;
103     }
104
105     printf("Num\tWhat\n");
106     while (cur != NULL) {
107         printf("%d\t%s\t0x%08x\n", cur->NO, cur->expr, cur->last_val);
108         cur = cur->next;
109     }
110 }

```

`list_watchpoints()` 遍历整条链表，输出编号、表达式和当前基线值，供 `info w` 使用。

还有检查监视点的 `check_watchpoints()`：

```

112  ✓ bool check_watchpoints(void) {
113      WP *cur = head;
114      bool triggered = false;
115
116  ✓  while (cur != NULL) {
117      bool success = true;
118      uint32_t new_val = expr(cur->expr, &success);
119  ✓  if (!success) {
120      printf("watchpoint %d evaluation failed: %s\n", cur->NO, cur->expr);
121      return true;
122  }
123
124  ✓  if (new_val != cur->last_val) {
125      printf("Watchpoint %d triggered: %s\n", cur->NO, cur->expr);
126      printf("Old value = 0x%08x (%u)\n", cur->last_val, cur->last_val);
127      printf("New value = 0x%08x (%u)\n", new_val, new_val);
128      cur->last_val = new_val;
129      triggered = true;
130  }
131
132      cur = cur->next;
133  }
134
135      return triggered;
136  }

```

`check_watchpoints()` 对 `head` 中每个监视点重新求值，与 `last_val` 对比。若发生变化，打印 old/new 值并更新 `last_val`。函数返回 `true` 表示应触发停机。

CPU 执行循环中的接入点在 `nemu/src/monitor/cpu-exec.c`：

```

31  ✓ #ifdef DEBUG
32      /* TODO: check watchpoints here.
33
34  ✓  if (check_watchpoints()) {
35      nemu_state = NEMU_STOP;
36  }
37  #endif

```

这段代码放在每条指令执行后，检查监视点，若触发就把 `nemu_state` 置为 `NEMU_STOP`，控制流会返回 `ui_mainloop()`，从而回到交互状态等待用户下一条命令。

4.3 调试器命令集成

在 `nemu/src/monitor/debug/ui.c` 中，将刚才的监视点相关的命令集成到 `cmd_table`。

- `w EXPR`：创建监视点；

- `d N`: 按编号删除监视点;
- `info w`: 列出当前监视点。

分别通过调用三个函数`cmd_w()`、`cmd_d()`、`cmd_info()`实现, 如下:

```
81  ∨ static int cmd_w(char *args) {
82  ∨    if (args == NULL) {
83      printf("Usage: w EXPR\n");
84      return 0;
85  }
86
87  ∨    while (*args == ' ') {
88      args++;
89  }
90  ∨    if (*args == '\\0') {
91      printf("Usage: w EXPR\n");
92      return 0;
93  }
94
95      bool success = true;
96      WP *wp = add_watchpoint(args, &success);
97  ∨    if (!success || wp == NULL) {
98      printf("Bad expression: %s\n", args);
99      return 0;
100  }
101
102      printf("Watchpoint %d: %s (0x%08x)\n", wp->NO, wp->expr, wp->last_val);
103      return 0;
104  }
```

```
106 static int cmd_d(char *args) {
107     if (args == NULL) {
108         printf("Usage: d N\n");
109         return 0;
110     }
111
112     while (*args == ' ') {
113         args++;
114     }
115
116     char *end = NULL;
117     long no = strtol(args, &end, 10);
118     if (*args == '\0' || *end != '\0' || no < 0) {
119         printf("Invalid watchpoint number: %s\n", args);
120         return 0;
121     }
122
123     if (delete_watchpoint((int)no)) {
124         printf("Watchpoint %ld deleted.\n", no);
125     }
126     else {
127         printf("No watchpoint number %ld.\n", no);
128     }
129
130     return 0;
131 }
```

```

54  ✓ static int cmd_info(char *args) {
55  ✓      if (args == NULL) {
56          printf("Usage: info [r/w]\n");
57          return 0;
58      }
59
60  ✓      if (strcmp(args, "r") == 0) {
61          printf("Registers:\n");
62          printf("eax: 0x%08x\n", cpu.eax);
63          printf("ecx: 0x%08x\n", cpu.ecx);
64          printf("edx: 0x%08x\n", cpu.edx);
65          printf("ebx: 0x%08x\n", cpu.ebx);
66          printf("esp: 0x%08x\n", cpu.esp);
67          printf("ebp: 0x%08x\n", cpu.ebp);
68          printf("esi: 0x%08x\n", cpu.esi);
69          printf("edi: 0x%08x\n", cpu.edi);
70          printf("eip: 0x%08x\n", cpu.eip);
71      }
72  ✓      else if (strcmp(args, "w") == 0) {
73          list_watchpoints();
74      }
75  ✓      else {
76          printf("Unknown info command '%s'\n", args);
77      }
78      return 0;
79  }

```

这三个函数有分别调用之前设计的 `add_watchpoint()`、`delete_watchpoint()`、`list_watchpoints()` 函数。

接下来验证测试一下：

```

❖ SuYaoLei@988d7f56373a:~/code/ics2017/nemu$ make run
./build/nemu -l ./build/nemu-log.txt
[src/monitor/monitor.c,47,load_default_img] No image is given. Use the default build-in image.
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 19:23:07, Apr 13 2026
For help, type "help"
(nemu) w 0x100025
[src/monitor/debug/expr.c,97,make_token] match rules[12] = "[0-9a-fA-F]+" at position 0 with len 8: 0x100025
Watchpoint 0: 0x100025 (0x00100025)
(nemu) w 1+9+8*2
[src/monitor/debug/expr.c,97,make_token] match rules[13] = "[0-9]+" at position 0 with len 1: 1
[src/monitor/debug/expr.c,97,make_token] match rules[6] = "+" at position 1 with len 1: +
[src/monitor/debug/expr.c,97,make_token] match rules[13] = "[0-9]+" at position 2 with len 1: 9
[src/monitor/debug/expr.c,97,make_token] match rules[6] = "+" at position 3 with len 1: +
[src/monitor/debug/expr.c,97,make_token] match rules[13] = "[0-9]+" at position 4 with len 1: 8
[src/monitor/debug/expr.c,97,make_token] match rules[8] = "*" at position 5 with len 1: *
[src/monitor/debug/expr.c,97,make_token] match rules[13] = "[0-9]+" at position 6 with len 1: 2
Watchpoint 1: 1+9+8*2 (0x0000001a)
(nemu) info w
Num      What
1        1+9+8*2 0x0000001a
0        0x100025      0x00100025
(nemu) d 1
Watchpoint 1 deleted.
(nemu) info w
Num      What
0        0x100025      0x00100025
(nemu)

```

这里能看到添加了两个监视点，第二个为表达式计算得到，能正常使用 `info w` 命令查看，并且删除了第二个监视点后，`info w` 命令也正确输出，说明实现是正确的。

4.4 断点

根据指导书的内容，断点我们可以通过如下的命令利用监视点实现，但是注意要在 `nemu/include/common.h` 中把 `DEBUG` 宏 开启：

```
w $eip == ADDR
```

其中ADDR为设置断点的地址，这样程序执行到ADDR的位置时就会暂停下来。测试如下：

```

SuYaoLei@988d7f56373a:~/code/ics2017/nemu$ make run
./build/nemu -l ./build/nemu-log.txt
[src/monitor/monitor.c,47,load_default_img] No image is given. Use the default build-in image.
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 14:58:43, Apr 16 2026
For help, type "help"
(nemu) w $eip == 0x10000a
[src/monitor/debug/expr.c,97,make_token] match rules[14] = "\$[a-zA-Z][a-zA-Z0-9]*" at position 0 with len 4: $eip
[src/monitor/debug/expr.c,97,make_token] match rules[0] = " +" at position 4 with len 1:
[src/monitor/debug/expr.c,97,make_token] match rules[1] = "==" at position 5 with len 2: ==
[src/monitor/debug/expr.c,97,make_token] match rules[0] = " +" at position 7 with len 1:
[src/monitor/debug/expr.c,97,make_token] match rules[12] = "0[xX][0-9a-fA-F]+" at position 8 with len 8: 0x10000a
Watchpoint 0: $eip == 0x10000a (0x00000000)
(nemu) c
[src/monitor/debug/expr.c,97,make_token] match rules[14] = "\$[a-zA-Z][a-zA-Z0-9]*" at position 0 with len 4: $eip
[src/monitor/debug/expr.c,97,make_token] match rules[0] = " +" at position 4 with len 1:
[src/monitor/debug/expr.c,97,make_token] match rules[1] = "==" at position 5 with len 2: ==
[src/monitor/debug/expr.c,97,make_token] match rules[0] = " +" at position 7 with len 1:
[src/monitor/debug/expr.c,97,make_token] match rules[12] = "0[xX][0-9a-fA-F]+" at position 8 with len 8: 0x10000a
[src/monitor/debug/expr.c,97,make_token] match rules[14] = "\$[a-zA-Z][a-zA-Z0-9]*" at position 0 with len 4: $eip
[src/monitor/debug/expr.c,97,make_token] match rules[0] = " +" at position 4 with len 1:
[src/monitor/debug/expr.c,97,make_token] match rules[1] = "==" at position 5 with len 2: ==
[src/monitor/debug/expr.c,97,make_token] match rules[0] = " +" at position 7 with len 1:
[src/monitor/debug/expr.c,97,make_token] match rules[12] = "0[xX][0-9a-fA-F]+" at position 8 with len 8: 0x10000a
Watchpoint 0 triggered: $eip == 0x10000a
Old value = 0x00000000 (0)
New value = 0x00000001 (1)
(nemu) █

```

能看到在 `0x10000a` 处设置断点后, 执行 `c` 后会在该位置中断, 断点功能触发正确。

问题：我们知道`int3`指令不带任何操作数, 操作码为1个字节, 因此指令的长度是1个字节. 这是必须的吗? 假设有一种`x86`体系结构的变种`my-x86`, 除了`int3`指令的长度变成了2个字节之外, 其余指令和`x86`相同. 在`my-x86`中, 文章中的断点机制还可以正常工作吗? 为什么?

这在架构实现不变的情况下是必须的, 单字节的好处在于调试器可以只替换目标地址处的一个字节, 不需要关心原指令的长度, 从而避免破坏后续指令的编码结构, 这使断点的插入和恢复都非常简单可靠。

如果在假想的 `my-x86` 中将 `int3` 改为 2 字节, 而其他机制保持不变, 那么原有断点实现无法直接正常工作, 因为断点需要覆盖指令开头的字节, 而 `x86` 是变长指令体系, 如果强行写入 2 字节断点指令, 就可能覆盖到下一条指令的一部分, 从而破坏指令流, 导致程序执行错误。此外, 断点触发后程序计数器的回退、原指令的恢复等逻辑, 也都依赖于断点指令长度, 如果仍按 1 字节处理, 就会出现偏移错误。

问题：如果把断点设置在指令的非首字节(中间或末尾), 会发生什么?

如果把断点设置在一条指令的非首字节 (中间或末尾), 程序可能出现非法指令异常、程序崩溃, 或者停在一个看似莫名其妙的位置。

原因在于 `x86` 是一种变长指令体系结构, `CPU` 在取指时总是从当前的指令边界 (即 `EIP` 指向的位置) 开始解码, 而不是按字节随意切分。一条指令在内存中虽然占多个字节, 但这些字节只有在从首字节开始解码时才有意义。如果你在中间某个字节插入断点, 就相当于破坏了原指令的编码结构。

而 `CPU` 仍然会从指令首字节开始执行, 但执行过程中读到被你修改的那个字节时, 整条指令的编码已经被篡改, 可能被解释成另一条完全不同的指令, 甚至是非法指令, 从而触发异常。而调试器本身也是基于断点设置在指令起始地址的假设来工作的, 它只保存和恢复首字节, 如果断点在中间字节, 就无法正确还原原指令, 也无法正确回退程序计数器。

问题：模拟器(Emulator)和调试器(Debugger)有什么不同? 更具体地, 和NEMU相比, GDB到底是如何调试程序的?

模拟器在软件中完整地模拟 CPU、寄存器、内存以及指令执行流程，程序并不是在真实硬件上运行，而是在模拟环境中被逐条解释执行。也正因为如此，模拟器天然掌控了程序执行的每一个细节，可以在任意时刻检查状态、插入监控逻辑（如 watchpoint），甚至改变执行行为。

而调试器（例如 GDB）并不负责执行程序，程序依然是在真实 CPU 上运行的。调试器的作用是借助操作系统和硬件提供的机制，对正在运行的程序进行控制和观察。比如单步执行则依赖 CPU 的 trap flag；查看变量、本质上是读取目标进程的内存。

5.i386手册

问题：假设你现在需要了解一个叫selector的概念,请通过i386手册的目录确定你需要阅读手册中的哪些地方

我应该查阅的内容如下：

CHAPTER 5	MEMORY MANAGEMENT.....	91
5.1	SEGMENT TRANSLATION.....	92
5.1.3	Selectors.....	96
CHAPTER 17	80386 INSTRUCTION SET.....	239
17.2	INSTRUCTION FORMAT.....	240
17.2.2	How to Read the Instruction Set Pages.....	246
17.2.2.11	Instruction Set Detail.....	255
	ARPL — Adjust RPL Field of Selector.....	263
Figures		
5-6	Format of a Selector	

四、必答题

问题1： 查阅i386手册理解了科学查阅手册的方法之后,请你尝试在i386手册中查阅以下问题所在的位置,把需要阅读的范围写到你的实验报告里面:

EFLAGS寄存器中的CF位是什么意思? ModR/M字节是什么? mov指令的具体格式是怎么样的?

EFLAGS寄存器中的CF位是什么意思?

查阅目录得到：

Figures	
2-8	EFLAGS Register
4-1	Systems Flags of EFLAGS Register

由 2-8 定位到 Page 34 of 421 的 2.3.4.1 Status Flags，得到 CF 是 Carry Flag，，继续检索 Carry Flag 到最后找到 Page 419 of 421，得到如下：

Carry Flag — Set on high-order bit carry or borrow; cleared otherwise.

ModR/M字节是什么?

查阅目录得到:

```
17.2 INSTRUCTION FORMAT.....240
17.2.1 ModR/M and SIB
Bytes.....241
```

然后即可定位。

mov指令的具体格式是怎么样的?

查阅目录得到:

```
17.2 INSTRUCTION FORMAT.....240
17.2.2 How to Read the Instruction Set
Pages.....246
17.2.2.11 Instruction Set
Detail.....255
MOV — Move Data.....345
MOV — Move to/from Special Registers.....347
```

然后即可定位。

问题2: shell命令--完成PA1的内容之后,nemu/目录下的所有.c和.h和文件总共有多少行代码?你是使用什么命令得到这个结果的?和框架代码相比,你在PA1中编写了多少行代码?

可以使用命令

```
git grep -nI '.' pa1 -- '*.c' '*.h' | wc -l
```

来统计总代码行数。

而要去除空行,就使用

```
git grep -nI '.' pa1 -- '*.c' '*.h' | wc -l
```

如下:

```
SuYaolei@988d7f56373a:~/code/ics2017/nemu$ git grep -nI '.' pa1 -- '*.c' '*.h' | wc -l
3312
```

而要求在 pa1 写了多少代码,只需要分别求得 pa1 和 pa0 的总代码行数,相减即可,如下:

```

● SuYaolei@988d7f56373a:~/code/ics2017/nemu$ git grep -nI '' pa1 -- '*.c' '*.h' | wc -l
4059
● SuYaolei@988d7f56373a:~/code/ics2017/nemu$ git grep -nI '' pa0 -- '*.c' '*.h' | wc -l
3487
○ SuYaolei@988d7f56373a:~/code/ics2017/nemu$

```

所以我总共写了 $4059 - 3487 = 572$ 行代码。

而要将这个命令加到meikefile里只需要在 makefile 中添加：

```

56 REF ?= HEAD
57 count:
58 | @git grep -nI '' $(REF) -- '*.c' '*.h' | wc -l
59

```

之后运行查看：

```

● SuYaolei@988d7f56373a:~/code/ics2017/nemu$ make count
4059

```

问题3：使用man打开工程目录下的Makefile文件,你会在CFLAGS变量中看到gcc的一些编译选项.请解释gcc中的-Wall和-Werror有什么作用?为什么要使用-Wall和-Werror?

gcc 的编译选项如下：

```

15 CFLAGS += -O2 -MMD -Wall -Werror -ggdb $(INCLUDES)

```

-Wall 的作用是开启常见的警告检查，比如变量未使用、可能未初始化就使用、类型不匹配、可疑的控制流写法等等。

-Werror 则是把警告视作错误，也就是只要出现警告，编译就直接失败，不能产出可执行文件。

使用这两个选项可以在编译阶段就尽可能消灭隐患，提升代码的健壮性和可维护性。

五、实验过程中遇到的问题与解决方法

在词法实现阶段，出现过问题就是同一个字符在不同语境下语义不同导致的误判，也就是 - 既可以是二元减法，也可以是一元负号，* 既可以是乘法，也可以是解引用。如果简单按字符直接定型， $1 + -1$ 会被错误切分为二元运算链， $--1$ 也会被错误理解为自减。最终解决办法就是在词法完成后进行一次语义重写，当 - 或 * 位于表达式开头，或其前一个 token 不是可结束子表达式的 token 时，分别改写为 TK_NEG 与 TK_DEREF，把歧义消除移到求值前。

第二个问题是在进行括号匹配的时候有关 check_parentheses() 的判定边界。最开始我认为只需检查首尾是否为括号即可，但这样会把 $(1)+(2)$ 误判成可整体剥壳，从而破坏表达式结构。但是后面修改为正确策略，增加提前一个闭合检查，若扫描过程中在 q 之前就回到 level==0，说明最外层括号并未包裹整个区间，应返回 false。

第三，在监视点当断点使用的实现上，代码里监视点触发条件是表达式值发生变化，也就是 new_val != last_val，因此当我输入 $w \$eip == 0x100008$ 但是不触发时，并不能立即判定监视点实现错误，还需要确认该

地址是否真的是指令入口，比如说，当我输入 x 扫描内存命令，然后把输出的按字对齐内存地址误当作 eip 实际取值序列，但 x86 是变长指令，eip 只会落在真实指令边界。解决方法也很简单，可以通过 si + 寄存器观察验证 eip 轨迹，比如：

```
SuYaoLei@988d7f56373a:~/code/ics2017/nemu$ make run
[src/monitor/monitor.c,30,welcome] Build time: 09:55:38, Apr 27 2026
For help, type "help"
(nemu) si
(nemu) p $pc
[src/monitor/debug/expr.c,97,make_token] match rules[14] = "\$[a-zA-Z][a-zA-Z0-9]*" at position 0 with len 3: $pc
1048581 (0x00100005)
(nemu) si
(nemu) p $pc
[src/monitor/debug/expr.c,97,make_token] match rules[14] = "\$[a-zA-Z][a-zA-Z0-9]*" at position 0 with len 3: $pc
1048586 (0x0010000a)
(nemu) si
(nemu) p $pc
[src/monitor/debug/expr.c,97,make_token] match rules[14] = "\$[a-zA-Z][a-zA-Z0-9]*" at position 0 with len 3: $pc
1048588 (0x0010000c)
(nemu) █
```

就可以在输出的这些地址处来设置断点。